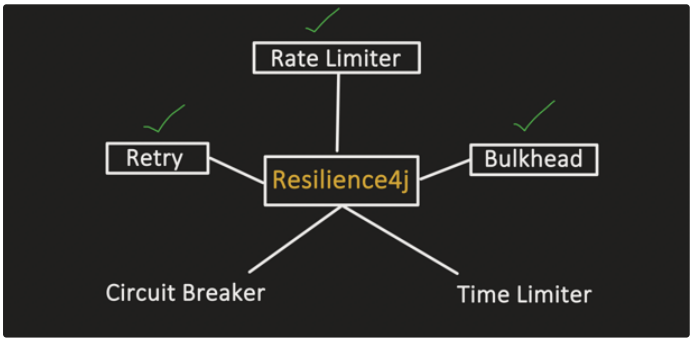
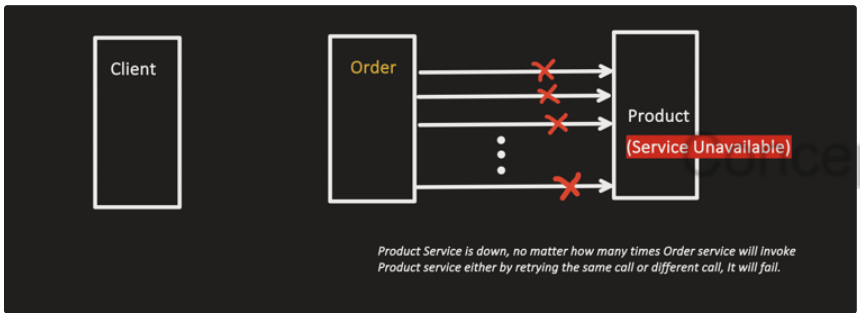
Circuit Breaker: Fault-Tolerant Microservice (Part-4)

To build Fault tolerant microservices: Resilience4j provides below mechanisms



Circuit Breaker:

- This pattern prevents an application to make repeated calls to a downstream service that is likely to fail.



Concept && Coding

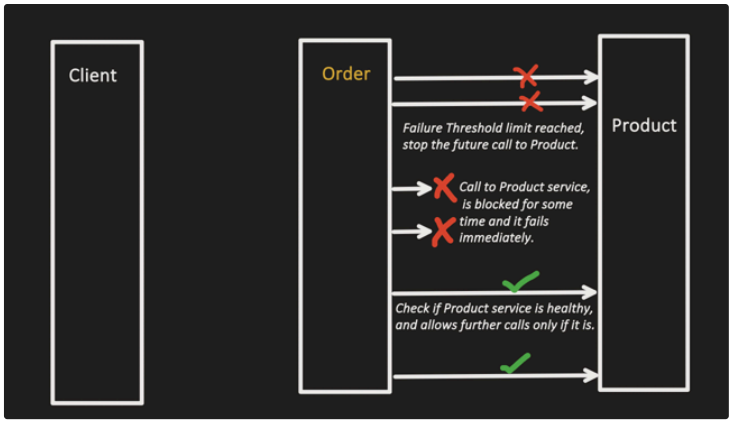
Disadvantages:

- It unnecessarily adds load to Product service and because of that Product service might take longer time to recover.
- Order service is unnecessarily wasting its resource (latency and thread blocking) by making call that is likely going to fail.

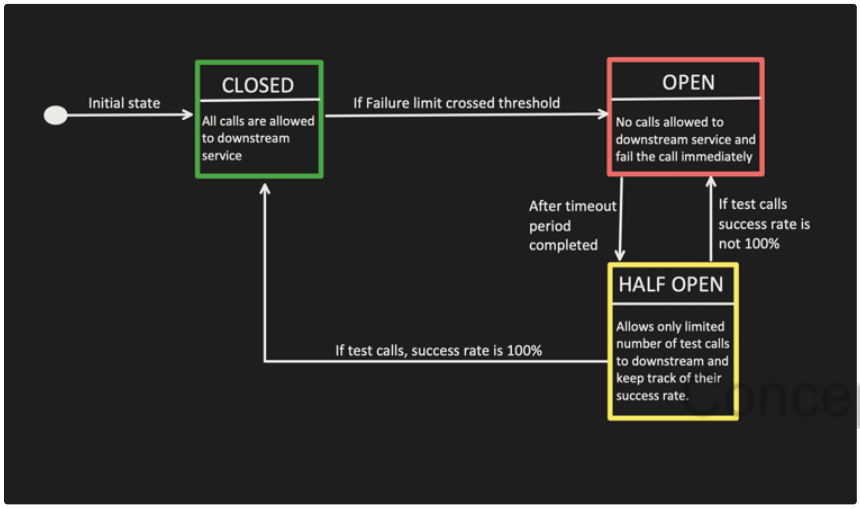
So, what's the solution?

How does Order service know, when to stop the downstream Product service call and when to restart?

Answer is "**Circuit Breaker**",again, lets revisit its definition.
It prevents an application to make repeated calls to a downstream service that is likely to fail.



States of Circuit Breaker:



Concept && Coding

Pom.xml dependency

```
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
    <version>2.1.0</version>
</dependency>

```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}

```

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
```

```
    String getProductById(@PathVariable("id") String id);
}

```

```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;
  /*
  Fallback method is invoked for each failure attempt.
  */

    @CircuitBreaker(name = "productService", fallbackMethod = "fallback")
    public void invokeProductAPI(String id) {
        productClient.getProductById(id);
    }

    public void fallback(Throwable ex) {
        System.out.println("not able to invoke product service");
    }
}

```

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#product service - circuit breaker configurations
resilience4j.circuitbreaker.instances.productService.sliding-window-type=COUNT_BASED
resilience4j.circuitbreaker.instances.productService.sliding-window-size=10
resilience4j.circuitbreaker.instances.productService.minimum-number-of-calls=5
resilience4j.circuitbreaker.instances.productService.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.productService.wait-duration-in-open-state=10s
resilience4j.circuitbreaker.instances.productService.permitted-number-of-calls-in-half-open-state=3
resilience4j.circuitbreaker.instances.productService.automatic-transition-from-open-to-half-open-enabled=true

```

Concept && Coding

sliding-window-type=COUNT_BASED

sliding-window-size=10

Tracks N (in this case 10) number of recent calls.

sliding-window-type=TIME_BASED

sliding-window-size=10s

Tracks calls made in a last N time duration(in this case 10sec)

minimum-number-of-calls = minimum no of call required in a window, before it start evaluating the failure rate.

failure-rate-threshold = circuit moves to OPEN state, once this threshold % limit reached.

wait-duration-in-open-state= time circuit stays in OPEN state, before it moves to HALF_OPEN state (automatically, if automatic-transition-from-open-to-half-open-enabled is set to true)

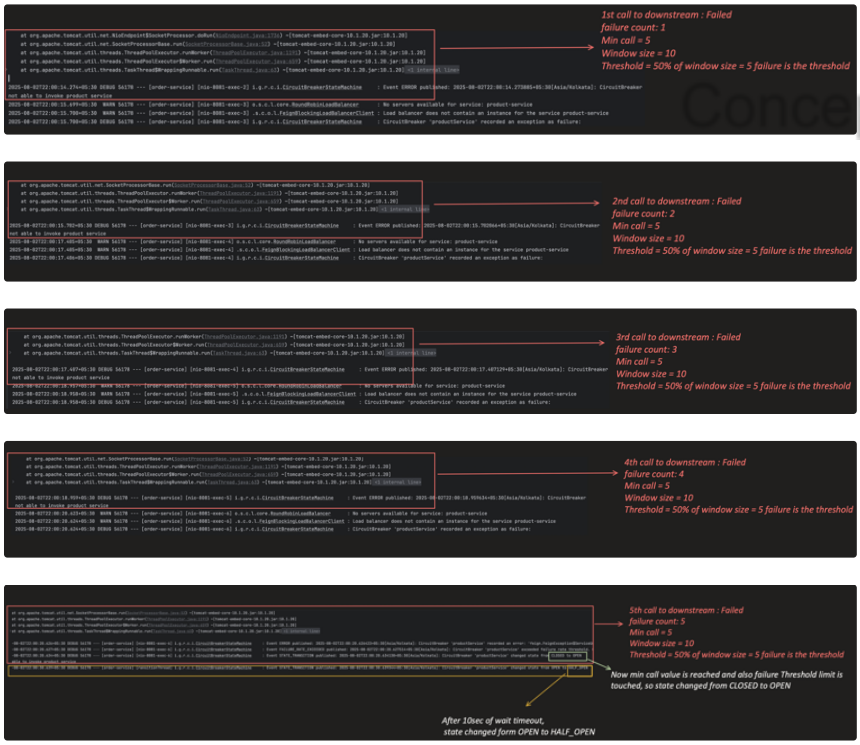
permitted-number-of-calls-in-half-open-state = no of test calls allowed in HALF_OPEN state to find out if downstream is healthy or not.

By default, the Circuit Breaker records all **RuntimeExceptions** and **Errors** as failures.

But if we want specific exception to be recorded and ignored, we can also configured it like below:

```
resilience4j.circuitbreaker.instances.productService.record-exceptions=java.io.IOException,org.springframework.web.client.HttpServerErrorException
resilience4j.circuitbreaker.instances.productService.ignore-exceptions=java.lang.IllegalArgumentException

```

Output:



Concept && Coding

Since max trial call limit reached, and success is not 100%, so status changed from HALF_OPEN to OPEN.

AOP intercept the call and pass it to "***CircuitBreakerStateMachine.java***" class.

Which has complete logic of changing one state to another, whenever there is a failure.

<u>Sample method from framework class:</u>

```
// Transitions to open state when thresholds have been exceeded.
//
// Params: result - the Result
private void checkIfThresholdsExceeded(Result result) {
    if (Result.hasExceededThresholds(result)
            && isClosed.compareAndSet(true, false)) {
  /*
    After every error, it checks if state need to be changed or not.
    Like here its checking, if threshold limit reached and state is CLOSED,
    then transit to OPEN state.

    Likewise similar method is present for different state with specific
    transition logic.
*/
        publishCircuitThresholdsExceededEvent(result, circuitBreakerMetrics);
        transitionToOpenState();
    }
}

```

Concept && Coding

One question, might be coming to you:

- Okay, for every failure call, we are checking if state need to be changed or not, make sense.
- But once it moved to OPEN state, then how automatically say after 10sec it move to HALF_OPEN state?

It uses, ScheduledThreadPoolExecutor

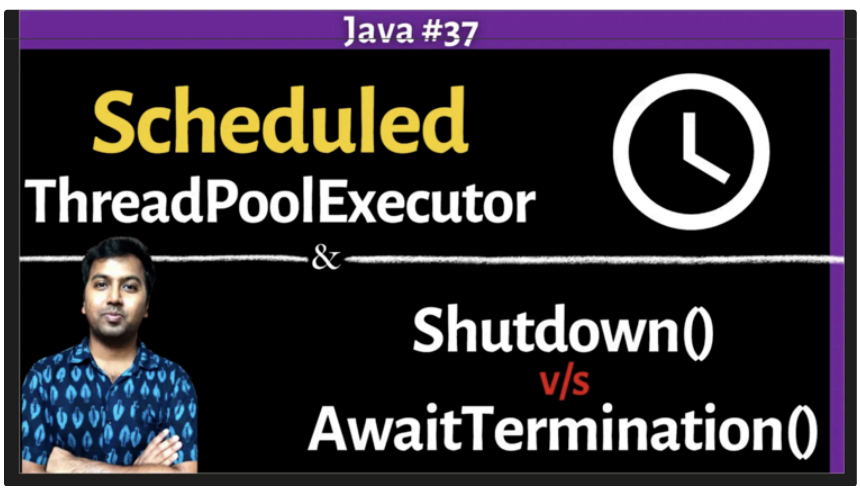
OpenState method from ***CircuitBreakerStateMachine.java*** framework class:

```
OpenState(final int attempts,
          final long waitDurationInMillis,
```

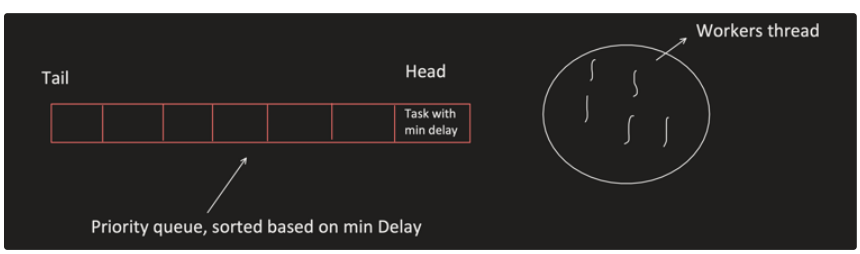
```java
            final Instant retryAfterWaitDuration,
            CircuitBreakerMetrics circuitBreakerMetrics) {

    this.attempts = attempts;
    this.retryAfterWaitDuration = retryAfterWaitDuration;
    this.circuitBreakerMetrics = circuitBreakerMetrics;

    if (circuitBreakerConfig.isAutomaticTransitionFromOpenToHalfOpenEnabled()) {
        ScheduledExecutorService scheduledExecutorService = schedulerFactory.getScheduler();
      /*
          It passes the request to ScheduledThreadPoolExecutor.
          1st parameter is the Task i.e. "toHalfOpenState"  to transit the state
          from OPEN to HALF_OPEN

          2nd parameter is the delay : like 10 or 20 or 30
          3rd parameter is Time Unit: second or minutes or millisecond
          */
      transitionToHalfOpenFuture = scheduledExecutorService
                .schedule(this::toHalfOpenState, waitDurationInMillis, TimeUnit.MILLISECONDS);
    } else {
        transitionToHalfOpenFuture = null;
    }

    isOpen = new AtomicBoolean(true);
}

```

I have already covered, how to use ScheduledThreadPoolExecutor in Java playlist. Pls check it out, if there is any doubt with its usage.

Concept && Coding



So internally **ScheduledThreadPoolExecutor** uses the concept of "DelayedQueue"



- Each available Worker thread, will look at the head of the Delayed Queue.
- If task delay is not yet expired, then thread waits(blocks) for that specific remaining delay period.
- Once the delay is over, OS wakes up the thread.
- One thread, pick the task from the Head and start executing it.
- Other thread start with the next task in the delayed queue, if task is not yet ready, then thread will wait(block) again.

Concept && Coding